

Benchmark di Tecnologie Big Data

Progetto 2 – Corso di Big Data (A.A. 2025/2026)

Alessandro Schmitt

Matricola: 577421

Repository GitHub: <https://github.com/AlessandroSchmitt/bigdata-flight-delay-analysis>

1 Introduzione e Obiettivi del Progetto

Il presente elaborato documenta la progettazione, l'implementazione e l'analisi sperimentale di una pipeline Big Data applicata all'elaborazione del dataset *Flight Delay Dataset – 2024*, contenente informazioni relative ai voli commerciali statunitensi, ai ritardi registrati e alle cancellazioni.

L'obiettivo principale del progetto consiste nel confrontare differenti paradigmi di elaborazione appartenenti all'ecosistema Apache, valutandone **espressività**, **semplicità implementativa**, **efficienza computazionale** e **scalabilità** al variare della dimensione dell'input e dell'ambiente di esecuzione.

Le tre tecnologie considerate sono:

- **Apache Spark Core (RDD):** paradigma di elaborazione a basso livello basato sui *Resilient Distributed Dataset*. L'utilizzo diretto delle trasformazioni RDD permette un controllo granulare sulle operazioni di mapping, aggregazione e gestione delle coppie chiave-valore, richiedendo tuttavia una maggiore quantità di codice applicativo.
- **Apache Spark SQL:** paradigma dichiarativo basato sul motore relazionale di Spark. I job sono implementati mediante query SQL esplicite, delegando al motore Spark SQL la generazione e l'ottimizzazione del piano di esecuzione.
- **Apache Hive:** infrastruttura di Data Warehousing distribuito basata sul linguaggio dichiarativo HiveQL. Le query sono eseguite attraverso **Apache Tez**, utilizzato come motore di esecuzione per le elaborazioni Hive del progetto.

Le tre implementazioni eseguono le medesime analisi utilizzando lo stesso dataset preprocessato e le stesse regole semantiche. Questa scelta consente di rendere il confronto il più possibile omogeneo, evitando che eventuali differenze nei risultati sperimentali siano dovute a differenti procedure di preparazione dei dati.

L'analisi comparativa è stata condotta in due distinti contesti infrastrutturali:

1. **Ambiente locale:** le implementazioni Spark Core e Spark SQL sono state eseguite in modalità `local[2]`, utilizzando due thread di elaborazione. Apache Hive è stato invece eseguito attraverso Apache Tez in ambiente containerizzato. Non è stata definita una modalità sperimentale separata in cui i job Spark vengono eseguiti tramite YARN sulla macchina locale.
2. **Ambiente Cloud distribuito (AWS EMR):** è stato utilizzato un cluster Amazon Elastic MapReduce composto da un nodo Primary e due nodi Core. I job Spark sono stati sottomessi al cluster tramite **YARN**, utilizzando la modalità `-master yarn -deploy-mode cluster`. Amazon S3 è stato utilizzato come sistema di storage per i dataset di input, gli output delle elaborazioni e i log prodotti durante gli esperimenti.

1.1 Analisi del Dataset Utilizzato

Il benchmark utilizza il dataset pubblico *Flight Delay Dataset – 2024*, disponibile sulla piattaforma Kaggle all’indirizzo <https://www.kaggle.com/datasets/hrishitpatil/flight-data-2024>. Il dataset originale contiene oltre sette milioni di record relativi ai voli effettuati negli Stati Uniti durante il 2024 e comprende **35 attributi** in formato CSV.

Le informazioni disponibili includono, tra gli altri elementi, il mese del volo, la compagnia aerea, l’aeroporto di origine, i ritardi in partenza e in arrivo, lo stato di cancellazione e le differenti categorie di cause associate ai ritardi o alle cancellazioni.

Prima dell’esecuzione dei job è stata realizzata una fase comune di preprocessing con l’obiettivo di costruire una rappresentazione canonica del dataset, utilizzata senza modifiche dalle tre tecnologie analizzate.

Al termine della preparazione, il dataset utilizzato per gli esperimenti è composto da:

- **7.061.582 record;**
- **12 attributi** rilevanti per le due analisi.

Le colonne mantenute nel dataset preprocessato sono:

```
month  
op_unique_carrier  
origin  
dep_delay  
arr_delay  
cancelled  
cancellation_cause  
carrier_delay  
weather_delay  
nas_delay  
security_delay  
late_aircraft_delay
```

Per analizzare la scalabilità delle tecnologie al crescere della quantità di dati, dal dataset completo sono state generate tre dimensioni progressive di input:

- **10%:** 703.895 record;
- **50%:** 3.528.413 record;
- **100%:** 7.061.582 record.

La selezione dei record utilizzati nelle frazioni è stata effettuata mediante un criterio deterministico, in modo da ottenere campioni riproducibili e garantire che Spark Core, Spark SQL e Hive elaborassero esattamente gli stessi dati per ciascuna dimensione di input.

Le decisioni relative alla riduzione delle colonne, alla gestione dei voli devianti, dei valori nulli e delle cause di ritardo sono descritte nel dettaglio nella Sezione 3.

1.2 Definizione dei Job di Analisi

Sono state implementate le due analisi previste rispettivamente dai punti **3.1** e **3.2** della specifica del progetto.

Job 1 – Statistiche delle compagnie aeree

Il primo job realizza un'aggregazione dei dati per ogni coppia composta da **compagnia aerea** e **aeroporto di partenza**.

Per ciascun gruppo vengono calcolati:

- il numero totale dei voli;
- il ritardo minimo all'arrivo;
- il ritardo massimo all'arrivo;
- il ritardo medio all'arrivo;
- il tasso di cancellazione;
- l'insieme ordinato dei mesi nei quali la compagnia ha operato nell'aeroporto considerato.

Il numero totale di voli e il tasso di cancellazione sono calcolati considerando tutti i record validi appartenenti al gruppo. Le statistiche relative al ritardo all'arrivo utilizzano invece esclusivamente i record per i quali `arr_delay` è disponibile.

Il tasso di cancellazione è rappresentato come rapporto compreso tra 0 e 1: ad esempio, il valore 0.0138 corrisponde a un tasso dell'1.38%.

La lista degli aeroporti prevista dalla specifica viene rappresentata in forma relazionale appiattita: l'output contiene quindi una riga per ogni coppia (*compagnia, aeroporto di partenza*), con le relative statistiche. Raggruppando le righe per compagnia si ricostruisce direttamente l'elenco degli aeroporti serviti.

Questo job permette di confrontare il modo in cui le tre tecnologie realizzano operazioni di raggruppamento, aggregazioni numeriche e costruzione di insiemi distinti.

Job 2 – Report dei ritardi per aeroporto e mese

Il secondo job produce un report per ogni coppia composta da **aeroporto di origine** e **mese**.

I voli non cancellati per i quali è disponibile il ritardo di partenza vengono suddivisi in tre fasce:

- **LOW**: ritardo in partenza inferiore a 15 minuti;
- **MEDIUM**: ritardo compreso tra 15 e 60 minuti, estremi inclusi;
- **HIGH**: ritardo superiore a 60 minuti.

Per ciascuna fascia vengono calcolati:

- il numero di voli;
- il ritardo medio in partenza;
- il ritardo medio in arrivo.

Le medie in partenza sono calcolate sui voli appartenenti alla relativa fascia. Le medie in arrivo utilizzano gli stessi voli, escludendo però dal calcolo gli eventuali record per i quali `arr_delay` è nullo.

Il job calcola inoltre le **tre cause di cancellazione o di ritardo più frequenti** per ogni coppia aeroporto-mese.

Per i voli cancellati viene utilizzata la causa di cancellazione associata al record. Per i voli non cancellati vengono invece considerate separatamente le cinque categorie di causa di ritardo presenti nel dataset:

- CARRIER;
- WEATHER;
- NAS;
- SECURITY;
- LATE_AIRCRAFT.

Una singola osservazione può contribuire a più cause quando più colonne di ritardo presentano contemporaneamente un valore positivo. Questa scelta evita di ridurre artificialmente l'informazione disponibile a una singola causa dominante e preserva la natura multi-causa dei dati originali.

La frequenza viene quindi interpretata come numero di **occorrenze volo–causa**: i minuti attribuiti a ciascuna causa non vengono utilizzati come peso della frequenza. Le categorie vengono ordinate per frequenza decrescente e, in caso di parità, secondo l'ordine alfabetico del nome della causa, in modo da ottenere un risultato deterministico. Per ogni coppia aeroporto–mese vengono infine selezionate le prime tre cause.

2 Stack Tecnologico

La progettazione e l'esecuzione della pipeline hanno richiesto la configurazione di un insieme di tecnologie appartenenti all'ecosistema Big Data, utilizzate sia per l'esecuzione locale sia per la successiva sperimentazione su cluster distribuito.

Poiché l'ambiente locale e Amazon EMR adottano distribuzioni software differenti, alcune versioni dei framework non coincidono esattamente tra i due contesti. Le implementazioni dei job e il formato dei dati sono tuttavia rimasti invariati, in modo da mantenere la stessa semantica delle elaborazioni nei due ambienti.

2.1 Framework dell'Ecosistema Big Data

I principali componenti utilizzati nel progetto sono i seguenti:

- **Java OpenJDK 17:** runtime Java utilizzato per l'esecuzione locale di Apache Spark. Nell'ambiente di sviluppo è stata utilizzata la distribuzione Temurin OpenJDK 17.0.20.
- **Apache Spark 3.5.5:** versione utilizzata nell'ambiente locale tramite PySpark. Spark è stato impiegato sia attraverso le API RDD di Spark Core, sia attraverso il motore Spark SQL.
- **Apache Hive 4.1.0:** versione utilizzata per gli esperimenti locali mediante l'immagine Docker ufficiale di Apache Hive. L'esecuzione delle query HiveQL è stata effettuata utilizzando Apache Tez come motore di elaborazione.
- **Apache Hadoop / YARN:** YARN è utilizzato come resource manager nell'ambiente AWS EMR. In ambiente locale i job Spark sono invece eseguiti direttamente mediante `local[2]`; non è stata definita una modalità sperimentale Spark-on-YARN locale separata.
- **Amazon EMR 7.13.0:** distribuzione utilizzata per l'ambiente Cloud. Il cluster è stato configurato con Apache Hadoop 3.4.2, Apache Spark 3.5.6, Apache Hive 3.1.3 e Apache Tez 0.10.2.

L'utilizzo di versioni leggermente differenti di Spark e Hive tra locale e Cloud è dovuto alla distribuzione software fornita da Amazon EMR. Le implementazioni Spark sono state mantenute compatibili tra le versioni 3.5.5 e 3.5.6 e gli stessi file HiveQL sono stati utilizzati sia nell'ambiente locale sia sul cluster.

2.2 Dipendenze Software e Strumenti di Supporto

Oltre ai framework Big Data, sono stati utilizzati alcuni strumenti di supporto per l'esecuzione, la riproducibilità e la successiva analisi sperimentale:

- **Python 3.12.1:** ambiente Python utilizzato nel Codespace per l'esecuzione degli script di preprocessing, validazione e benchmark.
- **PySpark 3.5.5:** interfaccia Python utilizzata per implementare ed eseguire localmente i job Spark Core e Spark SQL.
- **Docker:** utilizzato per isolare l'ambiente locale di Apache Hive e Tez, evitando di dover installare e configurare manualmente l'intero stack sul sistema host.
- **Matplotlib 3.11.1:** utilizzato per generare i grafici relativi ai tempi di esecuzione e al confronto tra ambiente locale e cluster EMR.
- **AWS CLI:** utilizzata per la gestione delle risorse Cloud, il caricamento dei dati su Amazon S3, la sottomissione degli step al cluster EMR e la raccolta automatizzata dello stato e dei tempi di esecuzione.
- **Git e GitHub:** utilizzati per il versionamento del codice sorgente, degli script di benchmark, dei risultati sperimentali e dei grafici prodotti. Le misurazioni ufficiali sono state associate al commit del codice effettivamente utilizzato per l'esecuzione, in modo da rendere riproducibile la configurazione sperimentale.

3 Preparazione e Pulizia dei Dati

La fase di preprocessing è stata progettata in modo indipendente dalle tre tecnologie analitiche, così da fornire a Spark Core, Spark SQL e Hive un unico dataset canonico con la stessa struttura e le stesse regole semantiche. In questo modo la preparazione dei dati viene eseguita una sola volta e non influenza direttamente il confronto dei tempi dei job.

Il dataset originale contiene 35 attributi, molti dei quali non sono necessari per le due analisi selezionate. La pipeline di preprocessing riduce quindi la dimensionalità dell'input, normalizza i valori rilevanti e applica un insieme comune di regole di validazione prima della fase sperimentale.

3.1 Decisioni di Progetto e Logica di Pulizia

Le principali decisioni adottate nella preparazione dei dati sono le seguenti:

1. **Selezione e normalizzazione degli attributi rilevanti:** dal dataset originale vengono utilizzati esclusivamente gli attributi necessari alla costruzione delle 12 colonne canoniche. I campi numerici vengono convertiti nei tipi appropriati, mentre i codici della compagnia aerea e dell'aeroporto di origine vengono ripuliti dagli spazi e normalizzati in maiuscolo. I valori mancanti nelle cinque colonne contenenti i minuti attribuiti alle cause di ritardo vengono rappresentati esplicitamente con valore zero.
2. **Validazione dei record e gestione dei voli devianti:** vengono mantenuti soltanto i record con mese compreso tra 1 e 12, indicatore di cancellazione valido e codici non vuoti per compagnia aerea e aeroporto di origine. I voli devianti vengono esclusi, poiché le relative statistiche di arrivo non risultano direttamente confrontabili con quelle dei voli regolarmente completati verso la destinazione prevista.

I voli cancellati vengono invece conservati, essendo necessari sia per il calcolo del tasso di cancellazione del Job 1 sia per la determinazione delle cause più frequenti del Job 2.

3. **Gestione dei ritardi e dei valori mancanti:** i valori negativi di `dep_delay` e `arr_delay` non vengono trattati come errori, poiché rappresentano rispettivamente partenze e arrivi anticipati. Analogamente, eventuali valori mancanti dei ritardi in partenza o in arrivo non vengono sostituiti artificialmente con zero. Le successive aggregazioni escludono tali valori soltanto quando la metrica specifica ne richiede la presenza.

4. **Rappresentazione delle cause di cancellazione e ritardo:** i codici di cancellazione del dataset originale vengono normalizzati secondo la seguente corrispondenza: $A \rightarrow \text{CARRIER}$, $B \rightarrow \text{WEATHER}$, $C \rightarrow \text{NAS}$ e $D \rightarrow \text{SECURITY}$.

Per i voli non cancellati non viene invece costruita una singola causa dominante. Le cinque colonne `carrier_delay`, `weather_delay`, `nas_delay`, `security_delay` e `late_aircraft_delay` vengono mantenute separatamente.

Questa scelta preserva l'informazione originale, poiché uno stesso volo può presentare minuti di ritardo attribuiti contemporaneamente a più categorie. Nel Job 2 un volo contribuisce pertanto a ciascuna causa per la quale il numero di minuti attribuiti è strettamente maggiore di zero.

Non viene eseguita una deduplicazione dopo la proiezione sulle colonne canoniche: due voli distinti nel dataset originale potrebbero infatti assumere gli stessi valori dopo l'eliminazione degli attributi non utilizzati, senza per questo rappresentare record duplicati.

Al termine della trasformazione viene prodotto un unico file CSV canonico, `flights_cleaned.csv`, contenente 7.061.582 record e le 12 colonne descritte nella Sezione 1.1.

La consolidazione in un singolo file viene effettuata esclusivamente durante il preprocessing e consente alle tre tecnologie di leggere la medesima rappresentazione fisica dei dati. Il tempo necessario a questa operazione non viene incluso nei benchmark dei job analitici.

3.2 Generazione delle Frazioni Progressive

Una volta costruito il dataset canonico completo, sono stati generati gli input utilizzati per lo studio sperimentale della scalabilità.

La generazione è automatizzata mediante uno script dedicato e utilizza un criterio deterministico di selezione dei record. In questo modo, ripetendo la procedura sul medesimo dataset canonico, vengono ottenuti gli stessi input e tutte le tecnologie lavorano esattamente sulle stesse osservazioni.

Sono state utilizzate tre dimensioni progressive:

- **10%:** 703.895 record, pari a circa 30 MB;
- **50%:** 3.528.413 record, pari a circa 151 MB;
- **100%:** 7.061.582 record, pari a circa 302 MB.

Le frazioni ridotte vengono ottenute a partire dal dataset preprocessato, mentre l'input al 100% coincide con il dataset canonico completo. La preparazione degli input viene effettuata prima dell'avvio della campagna sperimentale e non è quindi inclusa nei tempi di esecuzione riportati nei benchmark.

4 Analisi Sperimentale

L'analisi sperimentale ha l'obiettivo di confrontare le prestazioni delle tre tecnologie mantenendo invariati il contenuto degli input e la semantica dei job. Prima di descrivere il protocollo di benchmark vengono riassunte le principali scelte implementative adottate nelle tre soluzioni.

4.1 Scelte Implementative dei Job

Job 1

Le tre implementazioni utilizzano la stessa chiave logica (*compagnia, aeroporto di partenza*) e producono la stessa rappresentazione appiattita del risultato.

- **Spark Core:** ogni record viene trasformato in una coppia chiave-valore con chiave (`carrier, origin`). Il valore è un accumulatore contenente il numero di voli, il numero e la somma dei ritardi di arrivo non nulli, minimo e massimo, numero di cancellazioni e insieme dei mesi. Gli accumulatori vengono combinati mediante `reduceByKey`; al termine vengono calcolati media e tasso di cancellazione e viene ordinato l'insieme dei mesi.
- **Spark SQL:** il CSV canonico viene letto utilizzando uno schema esplicito e registrato come vista temporanea. Una query SQL esegue il `GROUP BY` su `op_unique_carrier` e `origin`, utilizzando `COUNT`, `MIN`, `MAX`, `AVG` e `SUM` per le statistiche numeriche. I mesi distinti vengono ottenuti tramite `COLLECT_SET`, ordinati e concatenati nella rappresentazione finale.
- **Hive/Tez:** il dataset canonico viene esposto tramite una tabella Hive e la stessa elaborazione è espressa in HiveQL mediante raggruppamento per compagnia e aeroporto e funzioni aggregate. La query viene eseguita dal motore Tez e produce lo stesso schema logico delle due implementazioni Spark.

In forma sintetica, la logica del Job 1 può essere rappresentata come:

```
input
-> key = (carrier, origin)
-> aggregazione di count, min, max, sum e cancellazioni
-> raccolta dei mesi distinti
-> calcolo media e cancellation rate
-> output
```

Job 2

Il secondo job richiede due elaborazioni logicamente distinte: il calcolo delle statistiche delle tre fasce di ritardo e il conteggio delle cause. I due risultati vengono successivamente combinati per la stessa chiave (*aeroporto, mese*).

- **Spark Core:** i record canonici vengono riutilizzati da due rami della pipeline. Il primo genera gli accumulatori relativi alle fasce LOW, MEDIUM e HIGH e li combina per chiave. Il secondo espande ogni record nelle relative occorrenze volo-causa, conta le frequenze per (`origin, month, cause`) e mantiene le tre cause migliori secondo l'ordinamento deterministico definito dal progetto. Le statistiche delle fasce e il Top-3 vengono infine combinati tramite join sulla chiave (`origin, month`).
- **Spark SQL:** il dataset viene registrato come vista temporanea e mantenuto in `MEMORY_AND_DISK`, poiché è utilizzato da due rami analitici. La query è organizzata mediante CTE. La CTE `band_stats` calcola conteggi e medie delle fasce usando aggregazioni condizionali. La CTE `cause_incidence` trasforma le cinque possibili cause in righe separate mediante `LATERAL VIEW EXPLODE`; seguono il conteggio delle frequenze e il ranking con `ROW_NUMBER()` partizionato per aeroporto e mese. Le prime tre cause vengono ricomposte in colonne e unite alle statistiche delle fasce mediante `LEFT JOIN`.
- **Hive/Tez:** la soluzione HiveQL adotta la stessa decomposizione relazionale. Le statistiche delle fasce sono espresse tramite aggregazioni condizionali, mentre le cause vengono espanse

in più righe e successivamente raggruppate. Una funzione di ranking assegna la posizione alle cause secondo frequenza decrescente e nome crescente; il Top-3 viene infine unito alle statistiche delle fasce tramite join.

La struttura logica può essere sintetizzata come:

```
                -> statistiche LOW/MEDIUM/HIGH --
input (origin,month)                -> join -> output
                -> cause -> frequenze -> Top-3 --
```

Le tre implementazioni adottano quindi paradigmi differenti, ma mantengono invariati chiavi, filtri, trattamento dei valori nulli, arrotondamenti e regole di ordinamento.

4.2 Protocollo di Benchmark

La campagna di benchmark è stata automatizzata mediante script dedicati, responsabili dell'esecuzione sequenziale dei job, della raccolta dei tempi e della produzione dei file di riepilogo utilizzati per i grafici.

Per ciascun ambiente sono state considerate tutte le combinazioni ottenute da:

- 2 job analitici;
- 3 dimensioni dell'input: 10%, 50% e 100%;
- 3 tecnologie: Spark SQL, Spark Core e Hive;
- 3 ripetizioni per ogni configurazione.

Ogni ambiente ha quindi prodotto $2 \times 3 \times 3 \times 3 = 54$ misurazioni, per un totale complessivo di **108 esecuzioni ufficiali** tra locale e cluster.

Le esecuzioni sono state effettuate in sequenza, evitando job concorrenti. Per ridurre l'influenza dell'ordine di esecuzione, nelle tre ripetizioni è stata utilizzata una rotazione delle tecnologie:

- **Run 1:** Spark SQL, Spark Core, Hive;
- **Run 2:** Spark Core, Hive, Spark SQL;
- **Run 3:** Hive, Spark SQL, Spark Core.

Per ogni configurazione viene utilizzata come misura principale la **mediana dei tre tempi di esecuzione**. La deviazione standard viene inoltre conservata e riportata nei grafici come indicatore della variabilità tra le ripetizioni.

Prima della campagna ufficiale, la correttezza delle implementazioni è stata verificata attraverso script di confronto dedicati. Gli output prodotti da Spark SQL, Spark Core e Hive sono stati normalizzati e confrontati, verificando l'equivalenza dei risultati prima di utilizzare i tempi per l'analisi prestazionale.

I tempi relativi al preprocessing e alla generazione delle tre frazioni non sono inclusi nelle misurazioni, poiché tali operazioni vengono eseguite una sola volta prima dell'avvio dei benchmark.

4.3 Ambiente Locale

Il primo contesto sperimentale è costituito dall'ambiente di sviluppo utilizzato tramite GitHub Codespaces.

La macchina disponibile dispone di **2 vCPU** e circa **7.8 GiB di memoria RAM**, senza memoria swap. Si tratta quindi di un ambiente single-node con risorse computazionali limitate rispetto al cluster utilizzato successivamente.

I job Spark Core e Spark SQL vengono avviati mediante `spark-submit` in modalità:

```
--master local[2]
```

Spark esegue quindi i task localmente sfruttando due thread di elaborazione.

Per Hive è stato predisposto un ambiente Docker basato su Apache Hive 4.1.0 con Apache Tez come motore di esecuzione. Le query HiveQL vengono sottomesse in sequenza sugli stessi input utilizzati dalle implementazioni Spark.

Gli input da 10%, 50% e 100% vengono letti dal file system locale. Il tempo di ciascuna esecuzione viene misurato attraverso un timer wall-clock ad alta risoluzione, utilizzando `time.perf_counter()` all'interno dello script di benchmark.

Poiché Spark e Hive utilizzano meccanismi di esecuzione differenti, i valori assoluti comprendono anche overhead specifici della tecnologia. L'analisi considera pertanto sia i tempi assoluti sia l'andamento delle curve al crescere dell'input.

4.4 Ambiente Cloud Distribuito (AWS EMR)

Il secondo contesto sperimentale utilizza un cluster reale distribuito tramite **Amazon EMR 7.13.0**.

Il cluster è composto da tre istanze EC2 di tipo **m5.xlarge**:

- **1 Primary Node**, responsabile del coordinamento del cluster;
- **2 Core Nodes**, utilizzati per l'esecuzione distribuita dei task.

Ogni istanza m5.xlarge dispone di 4 vCPU e 16 GiB di memoria. Le istanze sono state allocate in modalità On-Demand e il cluster utilizza YARN come resource manager.

I job Spark sono eseguiti mediante:

```
spark-submit --master yarn --deploy-mode cluster
```

Le query Hive vengono eseguite sullo stesso cluster mediante HiveQL e Apache Tez.

Amazon S3 viene utilizzato direttamente come storage per input, output, codice degli step e log del cluster. I job leggono direttamente gli input tramite URI `s3://` e salvano nello stesso modo i risultati, senza un trasferimento manuale preliminare verso HDFS.

Ogni esecuzione viene sottomessa ad Amazon EMR come uno *step* indipendente. La metrica principale utilizzata per il cluster è la differenza tra `StartDateTime` e `EndDateTime` dello step. Il tempo tra creazione ed effettivo avvio viene registrato separatamente come tempo di coda e non è incluso nella misura principale.

La **dimensione del cluster è stata mantenuta costante** per l'intera campagna sperimentale. Lo studio di scalabilità si concentra quindi sull'aumento della dimensione dell'input, mantenendo invariata la configurazione composta da un nodo Primary e due nodi Core. Questa scelta consente inoltre di rendere direttamente confrontabili tutte le misurazioni EMR effettuate.

5 Risultati

Prima dell’analisi prestazionale è stata verificata l’equivalenza funzionale delle tre implementazioni. I controlli effettuati non hanno evidenziato mismatch tra Spark SQL, Spark Core e Hive. Per questo motivo viene riportato un unico output rappresentativo per ciascun job, evitando di ripetere tre volte gli stessi risultati.

Sul dataset completo, il Job 1 produce 1.738 coppie compagnia–aeroporto, mentre il Job 2 produce 4.039 coppie aeroporto–mese. Le righe mostrate di seguito sono ordinate deterministicamente soltanto a scopo di visualizzazione.

Tabella 1: Prime 10 righe dell’output del Job 1 sul dataset completo.

Airline	Aeroporto	Voli	Min Arr.	Max Arr.	Media Arr.	Tasso Canc.	Mesi
9E	ABE	1015	-47.0	575.0	0.67	0.0138	01 02 03 04 05 06 07 08 09 10 11 12
9E	ABY	38	-32.0	321.0	4.37	0.0000	01 02 03 11 12
9E	AEX	877	-34.0	979.0	9.61	0.0103	01 02 03 04 05 06 07 08 09 10 11 12
9E	AGS	1602	-36.0	1091.0	6.97	0.0206	01 02 03 04 05 06 07 08 09 10 11 12
9E	ALB	1088	-51.0	394.0	-4.39	0.0193	01 02 03 04 05 06 07 08 09 10 11 12
9E	ATL	26798	-43.0	1207.0	2.46	0.0109	01 02 03 04 05 06 07 08 09 10 11 12
9E	ATW	175	-34.0	420.0	9.39	0.0000	01 04 05 06 07 08 09 10
9E	AUS	379	-35.0	1225.0	14.15	0.0053	01 02 03 04 05 06 07 11
9E	AVL	672	-43.0	949.0	1.92	0.0208	01 02 03 04 05 06 07 08 09 10 11 12
9E	AZO	14	-22.0	71.0	-2.93	0.0000	01

Per migliorare la leggibilità, le 17 colonne del Job 2 vengono rappresentate in due tabelle complementari riferite alle stesse dieci righe: la prima contiene le statistiche delle fasce, la seconda il Top-3 delle cause.

Tabella 2: Prime 10 righe del Job 2: statistiche delle fasce di ritardo.

Orig.	Mese	Low N	Low Dep.	Low Arr.	Med N	Med Dep.	Med Arr.	High N	High Dep.	High Arr.
ABE	1	275	-5.51	-14.56	30	34.90	32.50	30	241.30	234.03
ABE	2	296	-6.62	-19.57	19	28.53	10.11	14	227.93	217.93
ABE	3	339	-6.19	-18.37	28	30.68	19.89	23	173.43	163.39
ABE	4	303	-6.45	-17.30	29	33.45	27.66	29	258.66	250.38
ABE	5	320	-6.40	-12.50	26	30.08	29.58	39	183.13	177.13
ABE	6	335	-6.85	-16.46	37	26.65	17.22	37	173.24	172.32
ABE	7	280	-6.36	-10.59	41	34.61	30.15	37	138.08	137.08
ABE	8	289	-6.64	-14.06	42	32.02	27.98	27	146.96	144.30
ABE	9	276	-6.86	-15.90	24	38.08	32.88	16	265.06	261.25
ABE	10	302	-6.03	-17.90	31	26.23	11.97	18	193.17	183.83

Tabella 3: Prime 10 righe del Job 2: tre cause più frequenti.

Orig.	Mese	Causa 1	N1	Causa 2	N2	Causa 3	N3
ABE	1	NAS	46	CARRIER	32	LATE_AIRCRAFT	32
ABE	2	CARRIER	16	NAS	16	LATE_AIRCRAFT	12
ABE	3	LATE_AIRCRAFT	19	NAS	19	CARRIER	17
ABE	4	CARRIER	28	LATE_AIRCRAFT	26	NAS	21
ABE	5	NAS	36	LATE_AIRCRAFT	31	CARRIER	28
ABE	6	CARRIER	35	LATE_AIRCRAFT	32	NAS	21
ABE	7	CARRIER	54	LATE_AIRCRAFT	48	NAS	40
ABE	8	LATE_AIRCRAFT	37	NAS	36	CARRIER	33
ABE	9	LATE_AIRCRAFT	22	CARRIER	20	NAS	17
ABE	10	CARRIER	20	LATE_AIRCRAFT	20	NAS	13

5.1 Analisi Sperimentale in Ambiente Locale

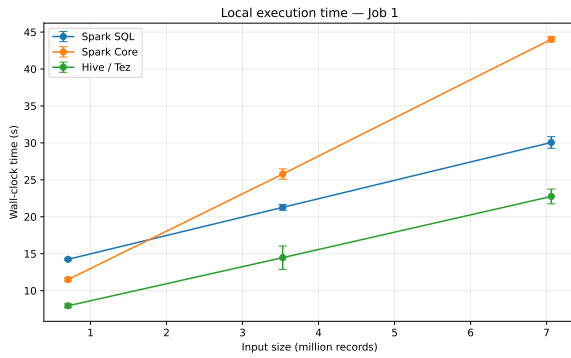
Le Tabelle 4 e 5 riportano le mediane dei tre tempi registrati per ciascuna configurazione locale.

Tabella 4: Tempi mediani di esecuzione locale per il Job 1, in secondi.

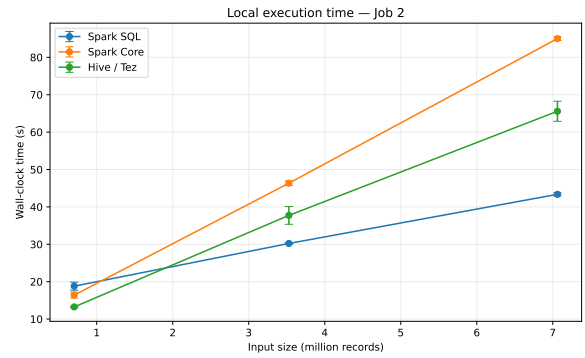
Dimensione	Spark SQL	Spark Core	Hive/Tez
10%	14.24	11.50	7.95
50%	21.27	25.78	14.46
100%	30.06	44.02	22.75

Tabella 5: Tempi mediani di esecuzione locale per il Job 2, in secondi.

Dimensione	Spark SQL	Spark Core	Hive/Tez
10%	18.79	16.40	13.24
50%	30.22	46.35	37.73
100%	43.36	85.02	65.58



(a) Job 1.



(b) Job 2.

Figura 1: Tempi di esecuzione locale delle tre tecnologie al variare della dimensione dell'input. Le barre rappresentano la deviazione standard delle tre ripetizioni.

Nel **Job 1**, Hive/Tez registra il tempo mediano più basso in tutte le dimensioni analizzate, passando da 7.95 secondi sul 10% a 22.75 secondi sul dataset completo. Spark Core risulta inizialmente più rapido di Spark SQL sul 10%, ma cresce più velocemente e raggiunge 44.02 secondi sul 100%. Spark SQL presenta invece un andamento più regolare, con 30.06 secondi sull'intero dataset.

Nel **Job 2** l'andamento cambia. Sul 10% Hive è la soluzione più rapida, con 13.24 secondi. All'aumentare dell'input Spark SQL mostra però la crescita più contenuta e diventa la tecnologia più veloce sia al 50% sia al 100%. Sul dataset completo Spark SQL richiede 43.36 secondi, Hive/Tez 65.58 secondi e Spark Core 85.02 secondi.

La maggiore crescita di Spark Core nel secondo job è coerente con la maggiore articolazione della pipeline RDD, che deve combinare statistiche delle fasce, conteggio delle cause e composizione finale del risultato.

5.2 Analisi Sperimentale in Ambiente Cloud (AWS EMR)

Le Tabelle 6 e 7 riportano i tempi mediani ottenuti sul cluster Amazon EMR.

Tabella 6: Tempi mediani di esecuzione su AWS EMR per il Job 1, in secondi.

Dimensione	Spark SQL	Spark Core	Hive/Tez
10%	32.07	32.06	32.07
50%	36.06	40.07	44.07
100%	40.06	50.06	46.08

Tabella 7: Tempi mediani di esecuzione su AWS EMR per il Job 2, in secondi.

Dimensione	Spark SQL	Spark Core	Hive/Tez
10%	34.06	34.06	36.06
50%	40.06	48.06	54.07
100%	46.07	68.06	54.06

Nel Job 1, sul 10% le tre tecnologie presentano tempi praticamente coincidenti, intorno ai 32 secondi, mostrando il peso del costo fisso di inizializzazione dello step distribuito quando l'input è contenuto.

Con l'aumento della dimensione dell'input le curve si separano. Spark SQL registra la mediana più bassa sia al 50%, con 36.06 secondi, sia sul dataset completo, con 40.06 secondi. Sul 100%, Hive/Tez richiede 46.08 secondi e Spark Core 50.06 secondi.

Anche nel Job 2 Spark SQL mostra l'andamento più favorevole sul cluster. Sul 10% Spark SQL e Spark Core risultano sostanzialmente equivalenti, mentre sul dataset completo Spark SQL termina in 46.07 secondi, Hive/Tez in 54.06 secondi e Spark Core in 68.06 secondi.

5.3 Confronto tra Ambiente Locale e Cluster

Le Figure 2 e 3 confrontano direttamente le mediane ottenute nei due ambienti.

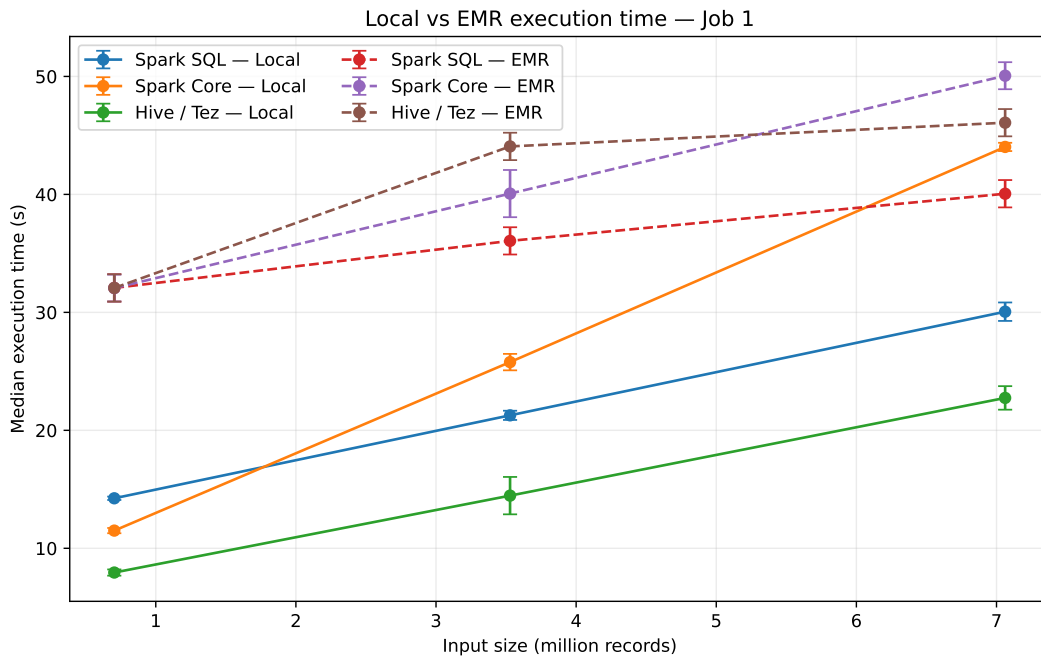


Figura 2: Confronto tra ambiente locale e AWS EMR per il Job 1.

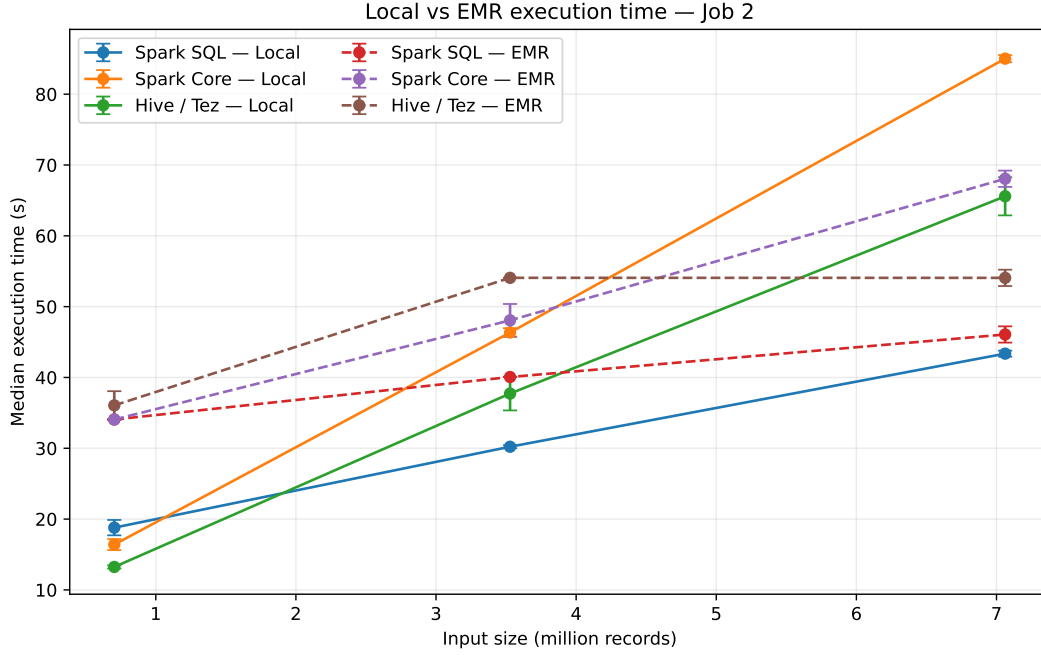


Figura 3: Confronto tra ambiente locale e AWS EMR per il Job 2.

Per il Job 1, l’ambiente locale mantiene tempi assoluti inferiori al cluster per tutte le tecnologie e per tutte le dimensioni considerate. Il dataset completo, pari a circa 302 MB, non è quindi sufficiente a compensare completamente l’overhead dell’esecuzione distribuita per questa elaborazione.

Il Job 2 mostra invece un comportamento differente sul dataset completo.

Tabella 8: Confronto Local–EMR sul dataset completo per il Job 2.

Tecnologia	Locale (s)	EMR (s)	Variazione
Spark SQL	43.36	46.07	+6.2%
Spark Core	85.02	68.06	-19.9%
Hive/Tez	65.58	54.06	-17.6%

Una variazione negativa indica una riduzione del tempo sul cluster. Spark Core riduce il tempo di circa il 19.9%, mentre Hive/Tez ottiene una riduzione di circa il 17.6%. Spark SQL, già particolarmente efficiente nell’ambiente locale, risulta invece circa il 6.2% più lento su EMR.

Il comportamento del cluster emerge ancora più chiaramente osservando la scalabilità. Passando dal 10% al 100% nel Job 2, Spark SQL cresce di circa 2.31 volte in locale e 1.35 volte su EMR; Spark Core passa da 5.18 a 2.00 volte e Hive/Tez da 4.95 a 1.50 volte.

L’esecuzione distribuita non garantisce quindi automaticamente il minor tempo assoluto: scheduling, inizializzazione, coordinamento e accesso allo storage introducono costi fissi particolarmente visibili sugli input più piccoli. L’andamento più contenuto delle curve EMR mostra tuttavia una migliore capacità di assorbire la crescita del volume dei dati, soprattutto nel Job 2.

Le differenze devono infine essere interpretate considerando la metodologia di misura: in locale viene registrato il tempo wall-clock dell’invocazione, mentre su EMR viene utilizzato l’intervallo `StartDateTime–EndDateTime` dello step. Le due metriche rappresentano entrambe il costo operativo dell’esecuzione, ma non sono identiche a livello di infrastruttura e wrapper.

6 Conclusioni

Il progetto ha consentito di confrontare Spark Core, Spark SQL e Hive/Tez applicando le stesse due analisi, gli stessi input e le stesse regole semantiche in ambiente locale e su un cluster distribuito reale.

I risultati non individuano una tecnologia migliore in assoluto. Nel Job 1 Hive/Tez ha ottenuto i tempi più bassi in locale, mentre sul cluster EMR Spark SQL ha registrato le mediane migliori sugli input al 50% e al 100%. Nel Job 2 Spark SQL ha mostrato la crescita più regolare sulle dimensioni medio-grandi, mentre sul dataset completo il passaggio al cluster ha ridotto il tempo di Spark Core e Hive/Tez rispettivamente di circa il 19.9% e il 17.6%.

Il confronto mette soprattutto in evidenza la differenza tra **tempo assoluto** e **scalabilità**. Per input relativamente contenuti il costo fisso del cluster può superare il vantaggio del parallelismo distribuito. All'aumentare del carico, però, le curve EMR crescono più lentamente, specialmente nel Job 2, indicando una migliore capacità di assorbire l'incremento del volume dei dati.

Dal punto di vista dell'**espressività e della semplicità implementativa**, Spark Core offre il maggiore controllo sulle trasformazioni, ma richiede una gestione esplicita di accumulatori, operazioni per chiave, ranking e join. Spark SQL permette invece di descrivere la stessa logica con query dichiarative e CTE, risultando più vicino alla formulazione analitica del problema. Hive/Tez offre un'impostazione analogamente dichiarativa, particolarmente naturale per aggregazioni, espansione di collezioni, funzioni di finestra e ranking.

Le operazioni di **shuffle e aggregazione** risultano più evidenti nella versione Spark Core, nella quale i raggruppamenti e le combinazioni per chiave sono espressi direttamente. In Spark SQL e Hive tali operazioni sono invece determinate dal piano fisico generato dai rispettivi motori. Il Job 2 richiede più fasi di aggregazione rispetto al Job 1 e mostra infatti differenze di scalabilità maggiormente marcate. I benchmark non consentono tuttavia di attribuire una singola differenza temporale a una specifica fase di shuffle, per cui le osservazioni vengono limitate al comportamento misurato complessivamente.

La **preparazione comune dei dati** ha inoltre un ruolo fondamentale nella comparabilità del benchmark. La riduzione da 35 a 12 attributi evita la lettura di informazioni non necessarie, mentre la normalizzazione garantisce che le tre tecnologie elaborino la stessa rappresentazione. Le cinque categorie di ritardo sono state mantenute separatamente anziché ridurle a una singola causa dominante, preservando così i casi in cui uno stesso volo presenta più cause positive. L'impatto prestazionale specifico del preprocessing non è stato misurato separatamente e non viene quindi quantificato.

Infine, la campagna è stata progettata con attenzione alla **riproducibilità**: gli input sono generati deterministicamente, ogni configurazione è eseguita tre volte con rotazione dell'ordine delle tecnologie, i risultati vengono salvati automaticamente e le misurazioni sono associate al commit Git del codice utilizzato. Il repository contiene codice sorgente, script di esecuzione, validazione, benchmark e grafici.

In conclusione, Spark Core rappresenta la soluzione con il maggiore controllo sulla computazione, Spark SQL offre nel progetto il miglior compromesso tra espressività, semplicità e crescita dei tempi, mentre Hive/Tez si dimostra competitivo nelle elaborazioni aggregate. Il passaggio a un cluster distribuito non produce automaticamente un vantaggio sui dataset più piccoli, ma diventa progressivamente più significativo quando il carico è sufficiente a compensare i costi di coordinamento e scheduling.